AO Studio

User Guide

Version 0.1.0

Visual IDE for Agent Orchestrator Workflows

Generated by AI Documenter — March 15, 2026

Table of contents

1 Introduction **3**
- 1.1 What is AO Studio? . . . 3
- 1.2 What is Agent Orchestrator? . . . 3
- 1.3 Who Is This For? . . . 3
- 1.4 Key Concepts . . . 3
  - 1.4.1 Teams . . . 3
  - 1.4.2 Agents . . . 4
  - 1.4.3 Workflow Phases . . . 4
  - 1.4.4 Governance Policies . . . 4
  - 1.4.5 Delegated Authority . . . 4
  - 1.4.6 Work Item Types . . . 4
  - 1.4.7 Condition Expressions . . . 4

2 Getting Started **4**
- 2.1 Prerequisites . . . 4
- 2.2 Starting Studio . . . 5
- 2.3 First Look . . . 5
- 2.4 The Sidebar . . . 6

3 Creating a Team **7**

4 Configuring Agents **7**
- 4.1 Adding an Agent . . . 8
- 4.2 Best Practices . . . 8

5 Designing the Workflow **9**
- 5.1 Phases Tab . . . 9
  - 5.1.1 Adding a Phase . . . 9
  - 5.1.2 Quality Gates . . . 9
- 5.2 Statuses Tab . . . 10
- 5.3 Graph Tab . . . 10

6 Setting Up Governance **10**
- 6.1 Delegated Authority . . . 10
- 6.2 Policies . . . 11

7 Defining Work Item Types **11**

8 Previewing Configuration **11**

9 Deploying to the Runtime **12**

10 Settings **13**

11 End-to-End Walkthrough **14**
- 11.1 Step 1: Create the Team . . . 14
- 11.2 Step 2: Add Three Agents . . . 14
- 11.3 Step 3: Design the Workflow . . . 14

11.4 Step 4: Configure Governance . . . 15
11.5 Step 5: Define Work Items . . . 15
11.6 Step 6: Preview and Validate . . . 15
11.7 Step 7: Deploy . . . 15
11.8 Step 8: Run the Pipeline . . . 15

12 Extension Stubs **15**

13 Troubleshooting **15**

1 Introduction

1.1 What is AO Studio?

AO Studio (Agent Orchestrator Studio) is a **visual design tool** for building multi-agent AI workflows. Instead of writing YAML configuration files by hand, you use a graphical interface to define agents, wire up workflow phases, set governance policies, and deploy everything to the Agent Orchestrator runtime with a single click.

Think of it as a visual IDE for AI pipelines — you design how your agents work together, and Studio generates all the configuration needed to run them.

1.2 What is Agent Orchestrator?

Agent Orchestrator is the runtime engine that actually executes your workflows. It manages:

- Routing work items through pipeline phases
- Calling LLM providers (OpenAI, Anthropic, Google, Grok, Ollama) for each agent
- Enforcing governance policies (auto-approve, escalate, deny)
- Logging audit trails of every decision
- Persisting state in PostgreSQL

Studio is the design tool. Agent Orchestrator is the execution engine. Studio generates YAML files that Agent Orchestrator reads to know what to do.

1.3 Who Is This For?

- **AI engineers** building multi-agent systems without writing boilerplate YAML
- **Team leads** designing agent workflows for specific business domains
- **Operations teams** configuring governance policies and thresholds
- **Developers** who prefer visual tools over text-based configuration

1.4 Key Concepts

1.4.1 Teams

A **team** is a complete profile that defines everything needed for a workflow. It contains agents, a workflow definition, governance rules, and work item types. You work on one team at a time in Studio.

1.4.2 Agents

An **agent** is an AI worker with a specific role. Each agent has:

- A **system prompt** that tells the LLM what to do (e.g., "You are a fraud detection specialist…")
- An **LLM configuration** (which provider, model, temperature)
- **Phase assignments** (which workflow steps this agent participates in)
- **Concurrency** settings (how many parallel instances can run)

1.4.3 Workflow Phases

Phases are the steps in your processing pipeline. Each phase runs one or more agents, has a defined execution order, and specifies what happens on success and failure (transition to another phase). Phases can have **quality gates** — conditions that must pass before proceeding.

1.4.4 Governance Policies

Policies are rules that control how work items are handled based on agent output. Actions include: **allow** (auto-approve), **deny** (reject), **escalate** (send to human), **review** (flag for inspection), **warn** (log warning, continue).

1.4.5 Delegated Authority

Three confidence thresholds that determine automatic decisions:

- **Auto-approve threshold** — Confidence above this means auto-approve
- **Review threshold** — Confidence below this means human review
- **Abort threshold** — Confidence below this means abort processing

1.4.6 Work Item Types

Work item types define the data schema for what your agents process. For example, an "Insurance Claim" work item has fields for claim text, policy number, claim amount, and claim type.

1.4.7 Condition Expressions

Conditions are used in quality gates and governance policies. Format: `<field> <operator> <value>`

Examples: `confidence >= 0.8`, `fraud_score < 0.2`, `category == 'hate_speech'`

Supported operators: `>=`, `<=`, `!=`, `==`, `>`, `<`, `in`

2 Getting Started

2.1 Prerequisites

- **Docker** and **Docker Compose** installed
- Port **8001** available (Studio) and optionally **8000** (Agent Orchestrator runtime)

2.2 Starting Studio

```
cd agent-orchestrator/studio
docker compose up -d --build
```

Studio is available at **http://localhost:8001**.

2.3 First Look

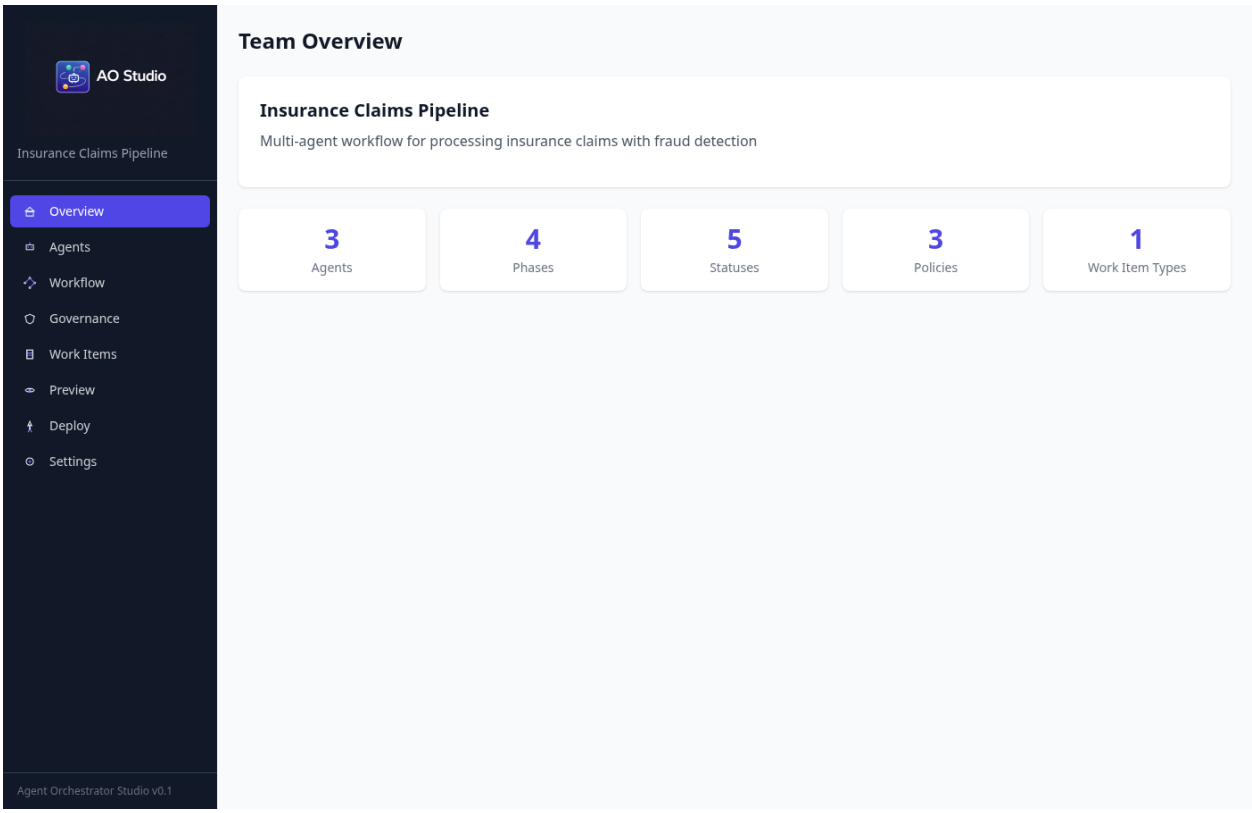

Figure 1: The Overview page is where you create a new team or import from a template.

The interface has a dark sidebar on the left for navigation and the main content area on the right.

2.4 The Sidebar

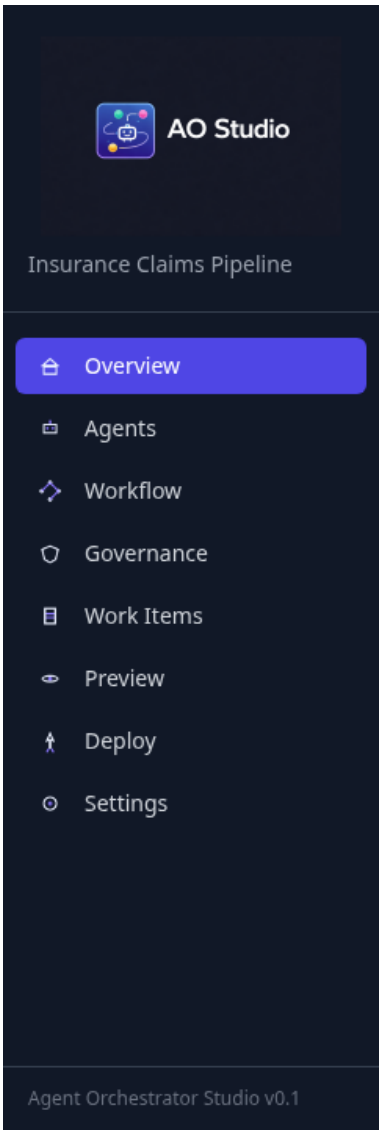

Figure 2: The sidebar provides navigation to all 8 sections of Studio.

Eight navigation items:

- **Overview** — Create team or import template
- **Agents** — Define AI agents with roles and LLM configs
- **Workflow** — Design the phase pipeline
- **Governance** — Set policies and confidence thresholds
- **Work Items** — Define domain-specific data schemas
- **Preview** — View and validate generated YAML
- **Deploy** — Write config files to the runtime
- **Settings** — Configure runtime connection

3 Creating a Team

On the Overview page, enter a **Team Name** and optional **Description**, then click **Create Team**.

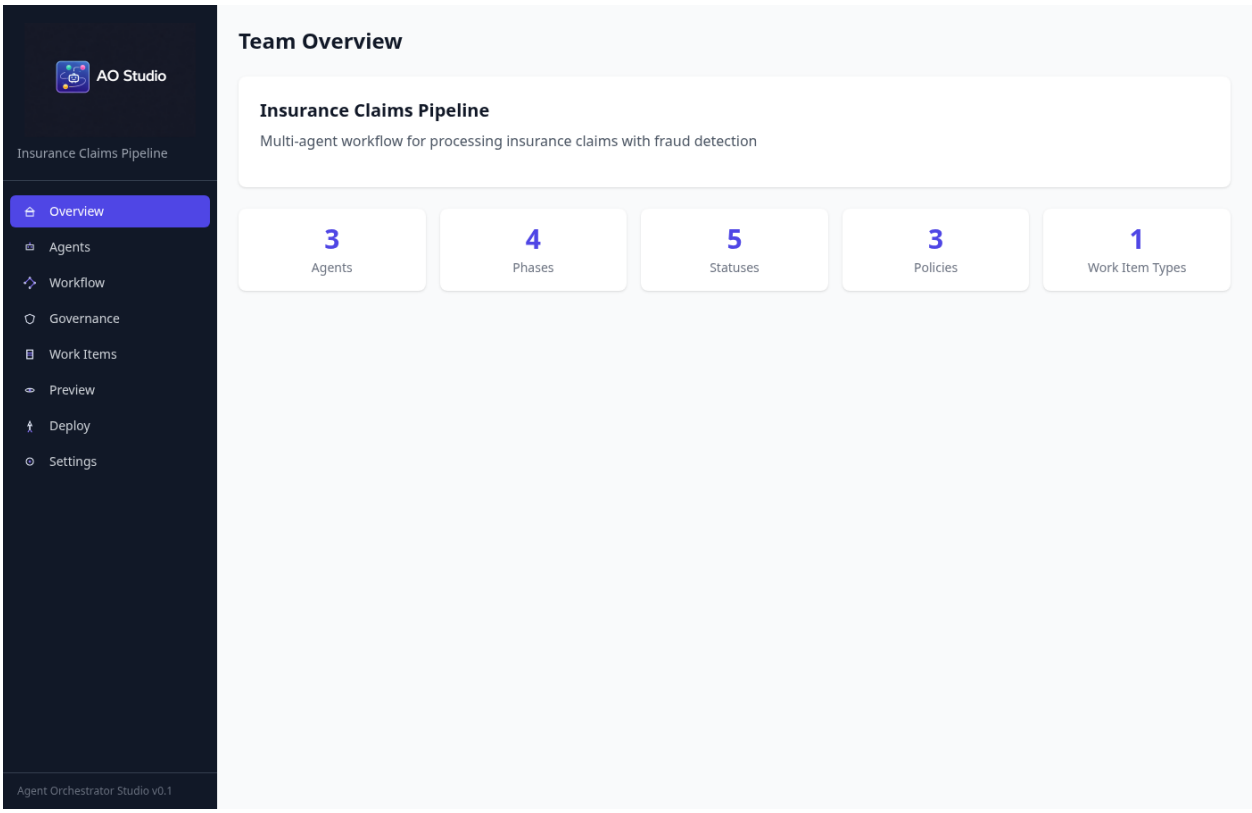

Figure 3: Create a new team or import an existing template.

Alternatively, import a built-in template:

- **content-moderation** — Content moderation pipeline with sentiment analysis
- **software-dev** — Software development lifecycle with 8 agents
- **research-team** — Research and analysis workflow

4 Configuring Agents

Navigate to **Agents** in the sidebar.

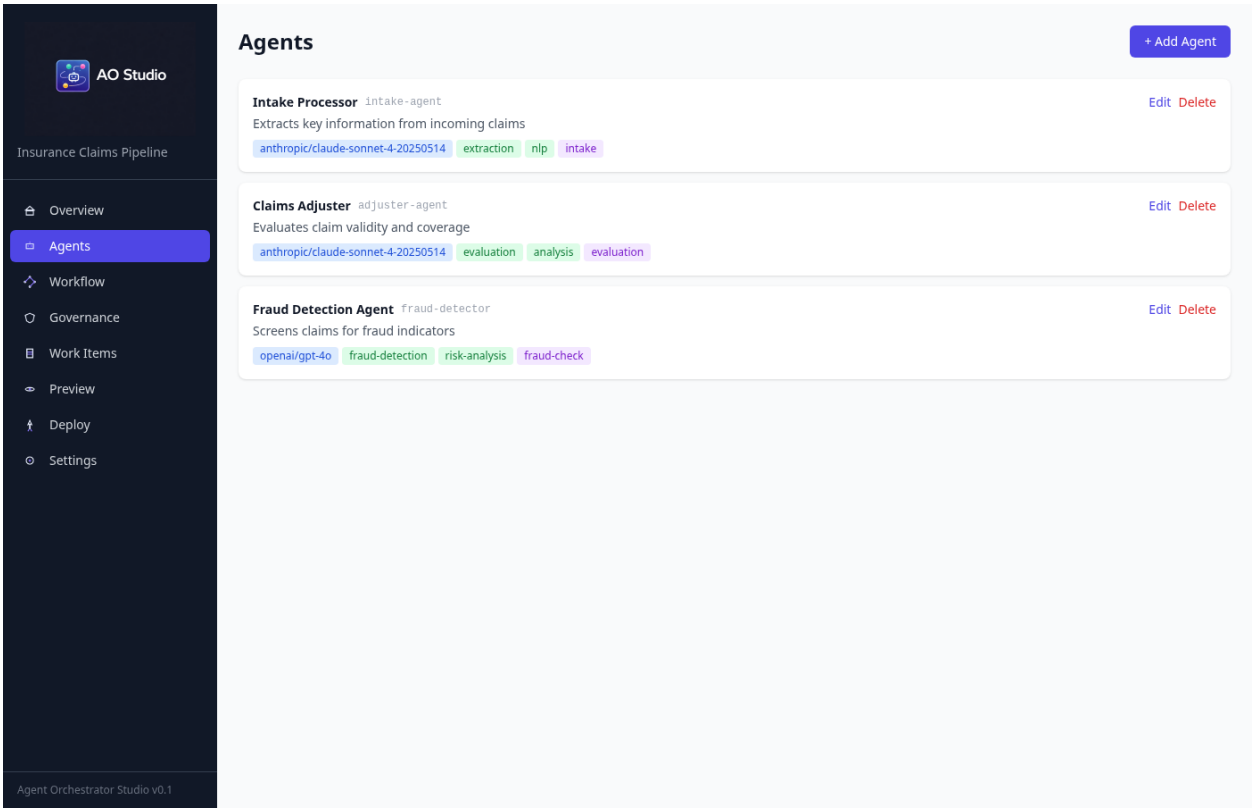

Figure 4: The Agents page shows all agents with their model, skills, and phase assignments.

Each agent card shows the name, ID, description, LLM provider/model badge, skill tags, and phase assignments.

4.1 Adding an Agent

Click **+ Add Agent**. The key fields:

- **ID** — Unique slug (e.g., `intake-agent`)
- **Name** — Human-readable name
- **System Prompt** — The instruction sent to the LLM defining the agent's behavior. Be specific about the role, what to analyze, what format to return, and what criteria to use.
- **Skills** — Comma-separated capability tags
- **Phases** — Which workflow phases this agent runs in
- **LLM Config**: Provider (`openai`, `anthropic`, `google`, `ollama`, `grok`), Model, Temperature (0.0–2.0), Max Tokens
- **Concurrency** — Parallel instances (1–100)

4.2 Best Practices

- **One responsibility per agent** — Do one thing well
- **Specific system prompts** — Exact instructions, not vague guidelines
- **Match model to task** — Powerful models for complex analysis, fast models for classification
- **Low temperature for analysis** (0.1–0.3), higher for creative tasks (0.5–0.8)

5 Designing the Workflow

Navigate to **Workflow** in the sidebar.

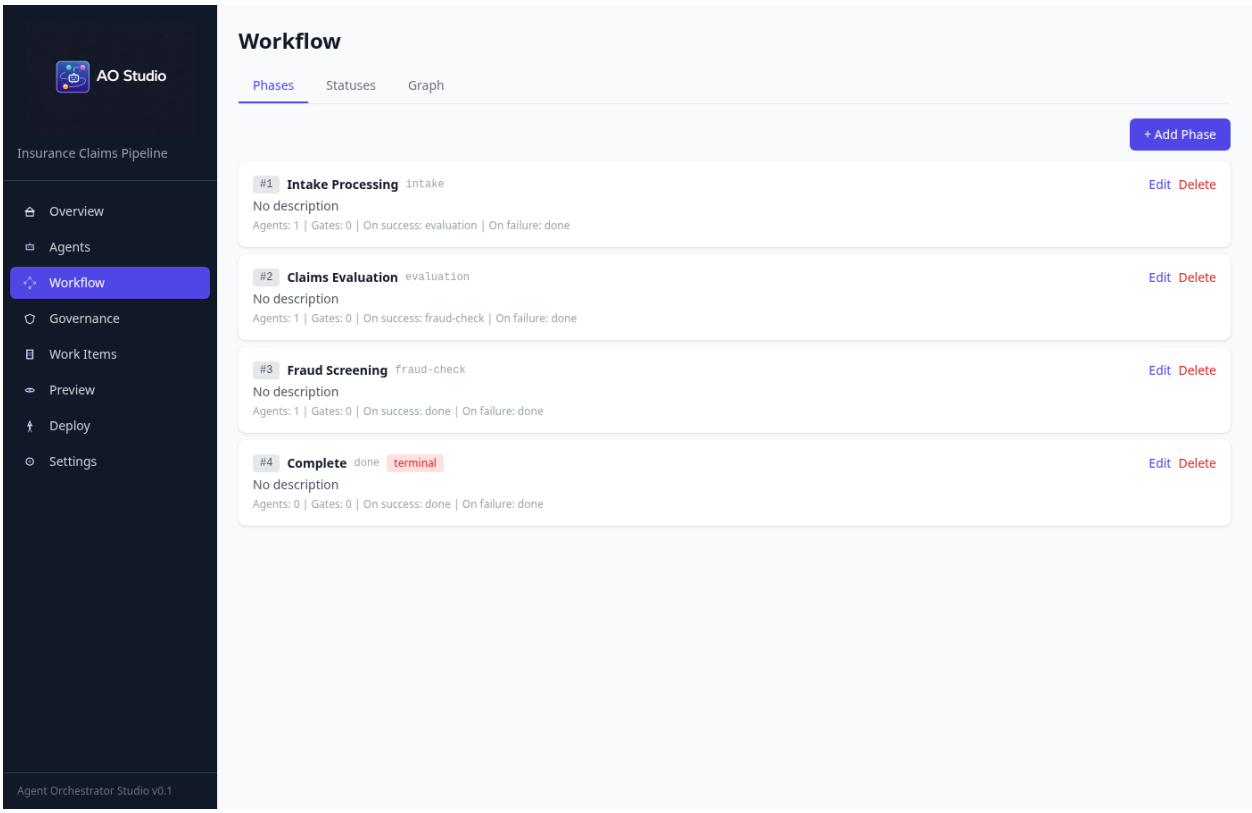

Figure 5: The Workflow page shows phases with execution order, transitions, and terminal markers.

5.1 Phases Tab

Each phase card shows order number, name, agent count, gate count, and transition targets.

5.1.1 Adding a Phase

Click **+ Add Phase**:

- **ID** — Unique slug
- **Order** — Execution sequence (1, 2, 3…)
- **Agents** — Which agents run in this phase
- **On Success** — Next phase if succeeded
- **On Failure** — Fallback phase if failed
- **Is Terminal** — Check for the final "done" phase
- **Requires Human** — Check if human approval needed

5.1.2 Quality Gates

Quality gates are checks after a phase completes. Each gate has conditions (e.g., `confidence >= 0.8`) and a failure action (`block`, `warn`, or `skip`).

5.2 Statuses Tab

Define work item lifecycle states (e.g., `submitted -> processing -> approved`). Mark one as `is_initial` and one or more as `is_terminal`.

5.3 Graph Tab

Visual DAG of phases with green (success) and red (failure) edges. Auto-validates for orphans and unreachable nodes.

6 Setting Up Governance

Navigate to **Governance** in the sidebar.

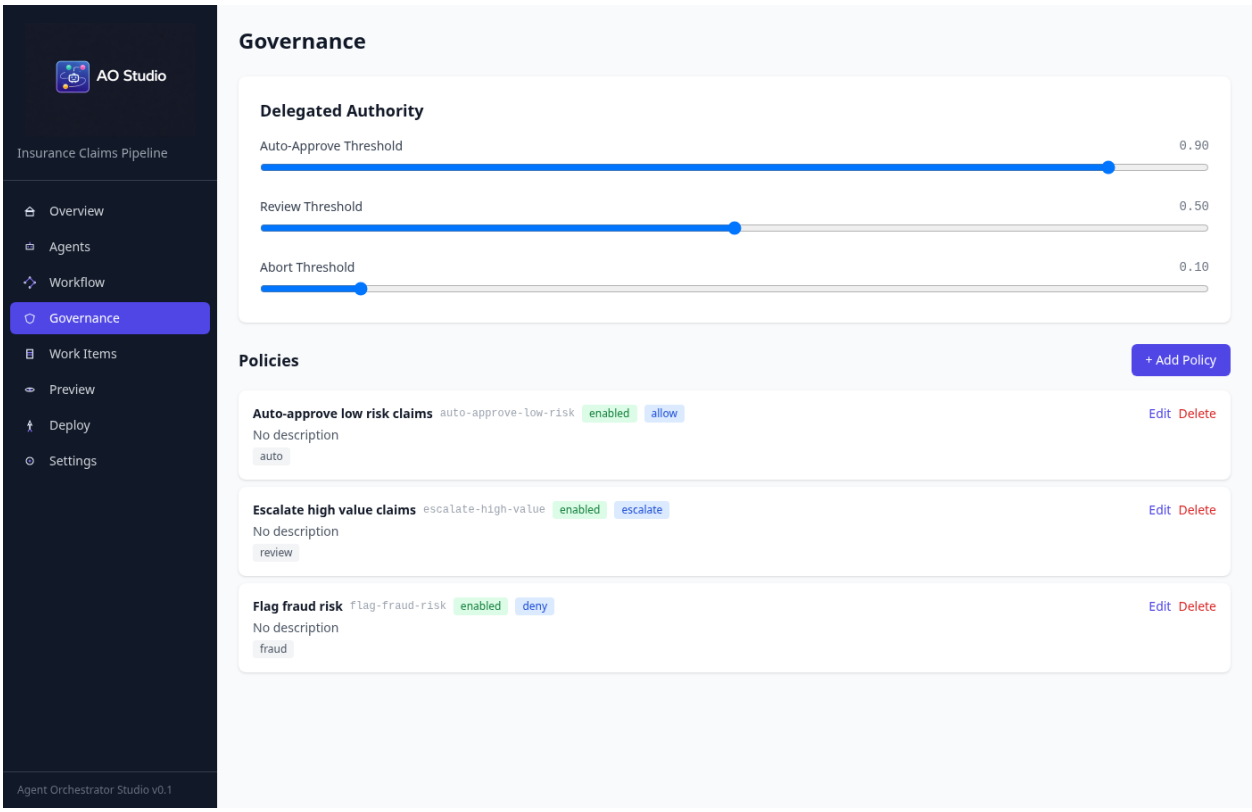

Figure 6: Governance with delegated authority sliders and ordered policy rules.

6.1 Delegated Authority

Three sliders control automatic decisions:

- **Auto-Approve** (0.9) — Confidence above this = auto-approve
- **Review** (0.5) — Confidence below this = human review queue
- **Abort** (0.1) — Confidence below this = abort

Thresholds must be in descending order.

6.2 Policies

Ordered rules evaluated by priority (highest first). First matching policy wins.

Policy	Action	Conditions
Auto-approve low risk	allow	`confidence >= 0.9`, `fraud_score < 0.2`
Escalate high value	escalate	`claim_amount >= 50000`
Flag fraud risk	deny	`fraud_score >= 0.8`

7 Defining Work Item Types

Navigate to **Work Items**.

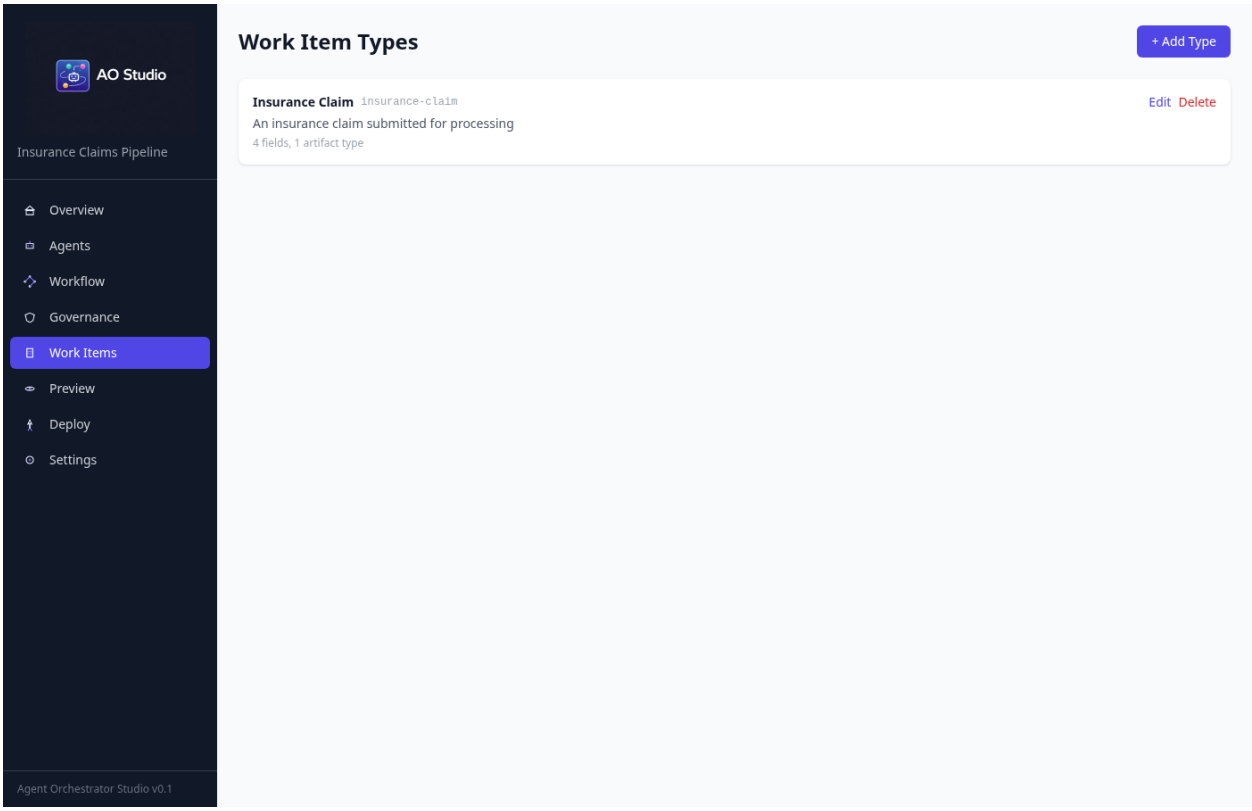

Figure 7: Work Item Types define the domain-specific data schema.

Add custom fields with name, type (`string`, `text`, `integer`, `float`, `enum`, `boolean`), and required flag. For `enum` fields, specify allowed values.

Add artifact types to define output files the workflow produces.

8 Previewing Configuration

Navigate to **Preview**.

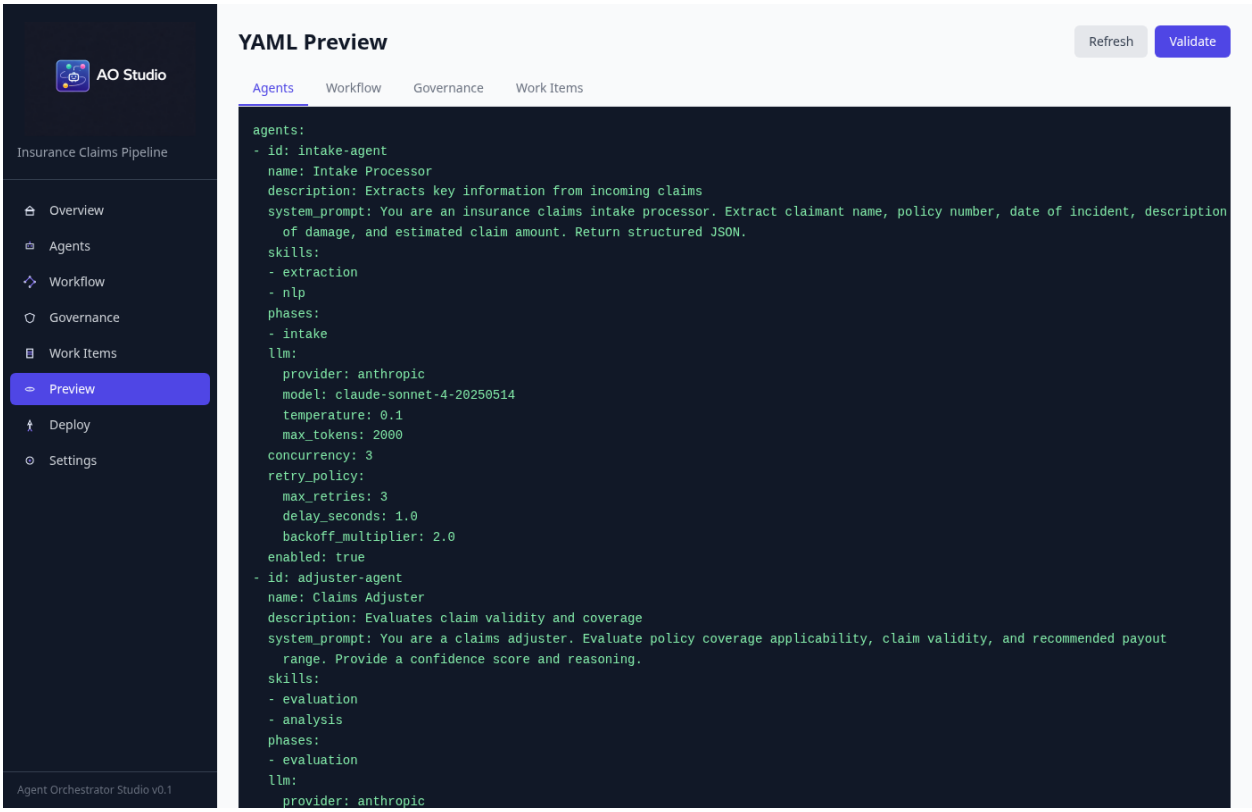

Figure 8: Preview shows generated YAML and validation results.

See generated YAML for `agents.yaml`, `workflow.yaml`, `governance.yaml`, `workitems.yaml`. Click **Validate** to check for errors before deploying.

9 Deploying to the Runtime

Navigate to **Deploy**.

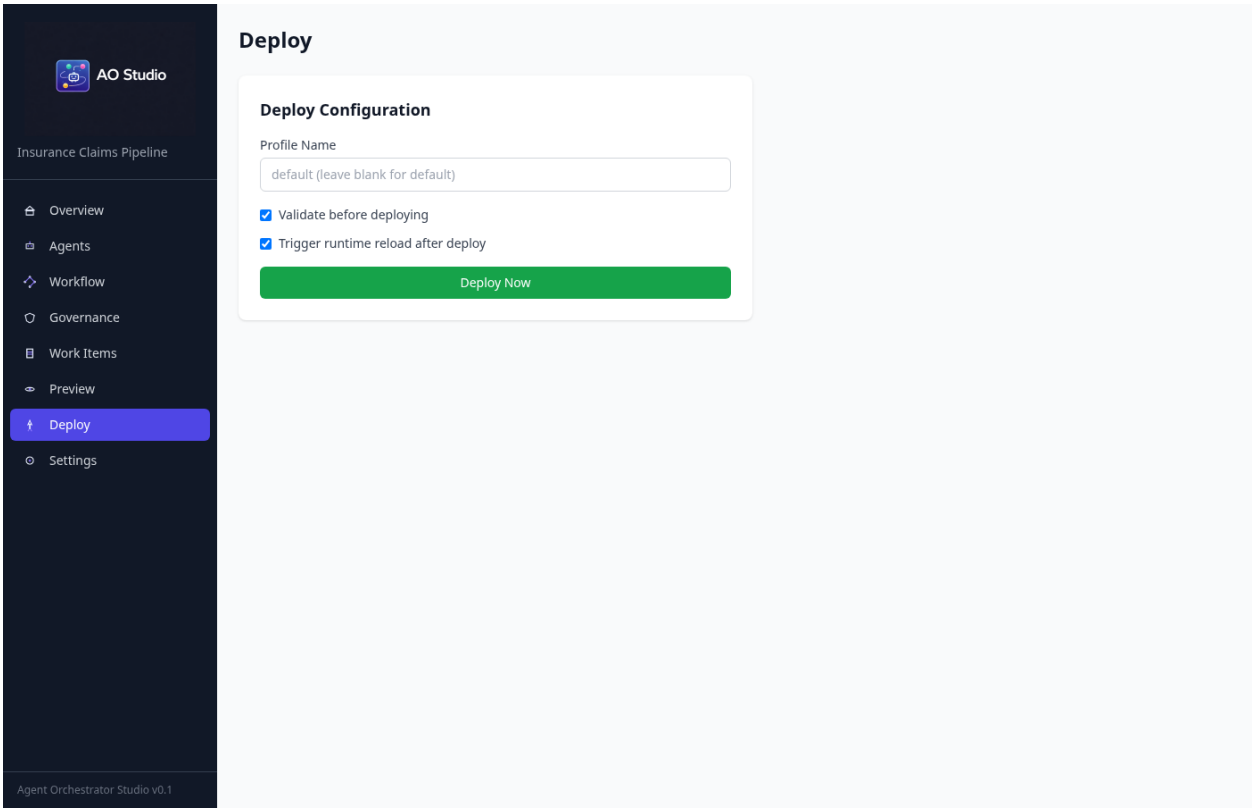

Figure 9: Deploy writes YAML files and optionally triggers a runtime reload.

Enter a **profile name**, enable validation and reload, click **Deploy**. This writes four YAML files to `workspace/profiles/{name}/` and optionally tells the runtime to switch profiles.

10 Settings

Navigate to **Settings**.

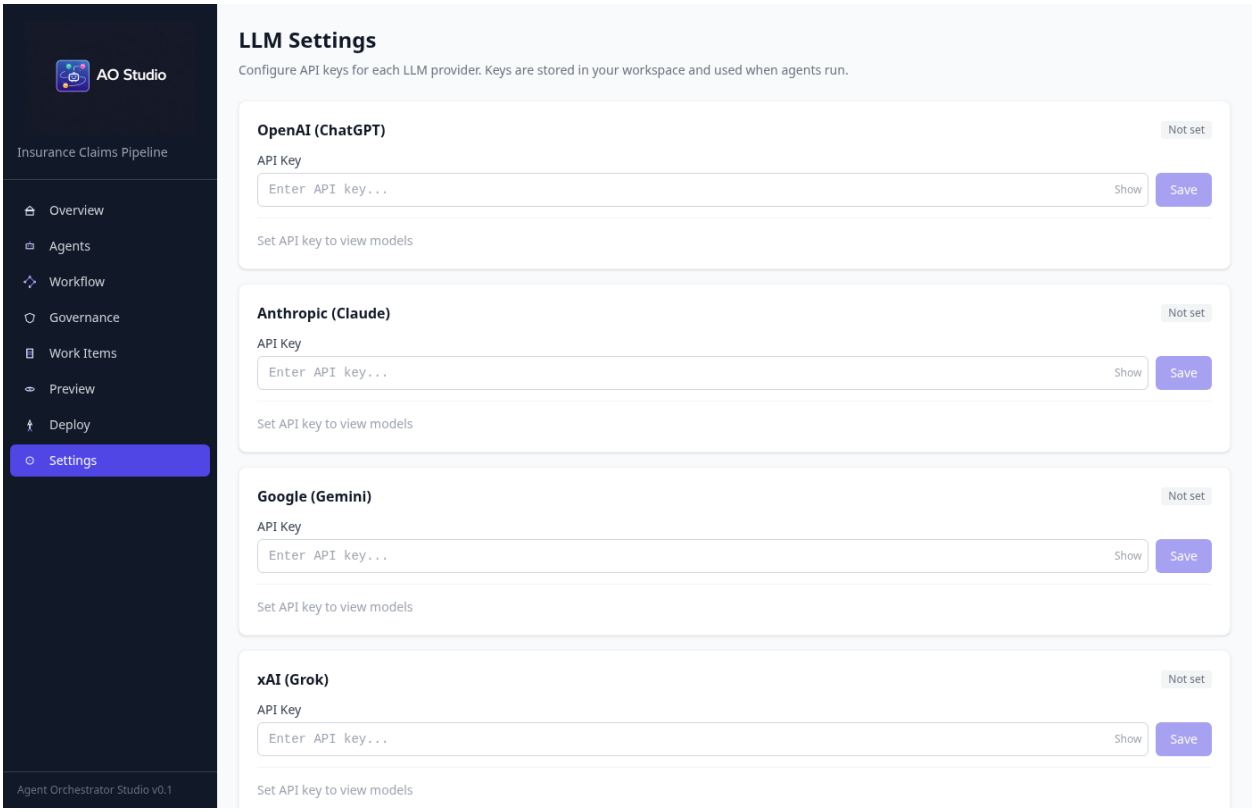

Figure 10: Settings configure the runtime connection URL.

Configure the **Runtime URL** (default: `http://localhost:8000`) used for connector discovery, validation, and deployment.

11 End-to-End Walkthrough

11.1 Step 1: Create the Team

Open **http://localhost:8001**. Enter team name "Insurance Claims Pipeline" and click **Create Team**.

11.2 Step 2: Add Three Agents

Navigate to **Agents**, click **+ Add Agent** for each:

- **Intake Processor** — Extracts claim info (Anthropic, Claude, temp 0.1, phase: intake)
- **Claims Adjuster** — Evaluates validity (Anthropic, Claude, temp 0.2, phase: evaluation)
- **Fraud Detector** — Screens for fraud (OpenAI, GPT-4o, temp 0.1, phase: fraud-check)

11.3 Step 3: Design the Workflow

Navigate to **Workflow**, add four phases: Intake -> Evaluation -> Fraud Screening -> Complete (terminal). Check the **Graph** tab to visualize the flow.

11.4 Step 4: Configure Governance

Navigate to **Governance**. Set thresholds (0.9/0.5/0.1). Add policies for auto-approve, escalate, and deny.

11.5 Step 5: Define Work Items

Navigate to **Work Items**. Create "Insurance Claim" with fields: claim_text, policy_number, claim_amount, claim_type.

11.6 Step 6: Preview and Validate

Navigate to **Preview**, click **Validate**. Fix any errors.

11.7 Step 7: Deploy

Navigate to **Deploy**, enter "insurance-claims", click **Deploy**.

11.8 Step 8: Run the Pipeline

```
curl -X POST http://localhost:8000/api/v1/execution/start
curl -X POST http://localhost:8000/api/v1/workitems \
  -H "Content-Type: application/json" \
  -d '{"id":"claim-001","type_id":"insurance-claim",
       "title":"Auto collision claim",
       "data":{"claim_text":"Rear-ended at a stoplight.",
               "policy_number":"P12345",
               "claim_amount":4500,"claim_type":"auto"}}'
curl http://localhost:8000/api/v1/workitems/claim-001
```

12 Extension Stubs

Studio generates Python code skeletons for extending your profile:

- **Connector Providers** — Integrate Slack, Jira, custom APIs
- **Event Handlers** — React to `work_item.submitted`, `phase.completed`
- **Phase Context Hooks** — Inject data before a phase runs

Stubs are marked **user-owned** after generation — Studio never overwrites them.

13 Troubleshooting

"No team loaded" — Create a team on the Overview page.

Templates not available — The Agent Orchestrator runtime must be running.

Validation errors — Check Preview page for specific messages (orphan phases, missing terminal phase, invalid agent references).

Runtime not reloading — Ensure runtime is running at the Settings URL.